



Manual del Programador

CMS de promoción turística de El Salvador · Proyecto de Cátedra, Fase 2

Integrante	Carné
Alberto Ramos Cruz	RC220772
Moises Alonso Marroquin Ayala	MA220150
Rene Eduardo Hernandez Castro	HC220857
Rafael Adolfo Ruiz García	RG210380

SAN SALVADOR, 30 DE SEPTIEMBRE DE 2026

Contenido

1. Introducción
2. Arquitectura del sistema
3. Tecnologías utilizadas
4. Estructura del código
5. Modelo de datos
6. Acceso a datos y persistencia
7. Seguridad
8. Manejo de errores y excepciones
9. Rendimiento y escalabilidad
- .0. Configuración del entorno de desarrollo
- .1. Guía de despliegue
- .2. Pruebas automatizadas
- .3. Flujo de trabajo con Git y GitHub
- .4. Cómo extender el sistema

1. Introducción

Este manual describe la arquitectura, la base de código y los procedimientos técnicos de **Trips SV**, un sistema de gestión de contenidos (CMS) para la promoción del turismo en El Salvador. Está dirigido a desarrolladores que necesiten mantener, desplegar o ampliar el sistema.

1.1 Descripción del sistema

El sistema reconoce tres niveles de acceso:

Rol	Capacidades
Visitante anónimo	Explora categorías y destinos, busca con filtros y lee calificaciones, reseñas y conversaciones de la comunidad, sin poder publicar.
Usuario registrado	Todo lo anterior, más calificar destinos de 1 a 5 estrellas, publicar reseñas, responder a otros usuarios, guardar favoritos y planificar itinerarios de viaje día por día.
Administrador	Todo lo anterior, más la gestión de categorías y destinos (con imagen y contenido enriquecido) y la moderación de comentarios: ocultarlos, volver a mostrarlos o eliminarlos.

2. Arquitectura del sistema

Trips SV sigue el patrón **MVC (Modelo–Vista–Controlador)** de ASP.NET Core, organizado en **capas lógicas** (N-capas) dentro de un mismo proyecto web: presentación, negocio y datos. Cada capa solo conoce a la capa inmediatamente inferior a través de interfaces.

Presentación

Controllers · Areas/Administracion · Views (Razor) · ViewModels · wwwroot

Recibe solicitudes HTTP, valida la entrada, aplica autorización y arma la respuesta HTML.

▼ depende de interfaces

Negocio

Servicios (ICategoriasServicio, ISitiosServicio, IComentariosServicio, IPuntuacionesServicio, IFavoritosServicio, ITinerariosServicio, ISanitizadorHtml)

Contiene las reglas del dominio y devuelve un **Resultado** con éxito o mensaje de error.

▼ usa

Datos

Modelos · Datos/ContextoTripSV · Datos/Configuraciones · Datos/Migraciones

Entity Framework Core mapea las entidades a SQL Server mediante Fluent API y migraciones Code-First.

▼

SQL Server

Base de datos TripSV

2.1 Justificación

- **Renderizado del lado del servidor.** Las páginas públicas del catálogo deben cargar rápido y ser indexables. MVC con Razor genera HTML completo en el servidor sin depender de un cliente pesado en JavaScript.
- **Formularios con validación y protección integradas.** El sistema es intensivo en formularios (reseñas, calificaciones, administración). MVC ofrece enlace de modelos, validación con DataAnnotations en servidor y cliente, y tokens antifalsificación sin trabajo adicional.
- **Controladores delgados y reglas testeables.** Al concentrar la lógica en servicios, las reglas críticas se prueban sin levantar el servidor web (ver sección 12).
- **Separación por rol.** El módulo de administración vive en un *Área* propia con autorización a nivel de clase, lo que evita exponer por error una acción administrativa.
- **Alternativas descartadas.** MVVM corresponde a clientes de escritorio o SPA con enlace de datos bidireccional, innecesario aquí. Una arquitectura de microservicios agregaría complejidad operativa

(varios despliegues, comunicación entre servicios) desproporcionada para el tamaño del dominio y del equipo.

2.2 Flujo de una solicitud

Ejemplo: un usuario califica un destino.

- 1 El navegador envía `POST /Puntuaciones/Calificar` con el token antifalsificación.
- 2 `PuntuacionesController` exige sesión iniciada (`[Authorize]`) y valida el token (`[ValidateAntiForgeryToken]`).
- 3 El controlador llama a `IPuntuacionesServicio.CalificarAsync` , que valida el rango 1–5, crea o actualiza la calificación del usuario y recalcula el promedio del sitio.
- 4 El servicio devuelve un `Resultado` ; el controlador guarda el mensaje en `TempData` y redirige al detalle del sitio (patrón Post/Redirect/Get), evitando reenvíos duplicados al recargar.

2.3 Patrones de diseño y principios SOLID

Patrón o principio	Aplicación en Trips SV
Capa de servicios	Un servicio por módulo del dominio encapsula las reglas de negocio.
Inyección de dependencias	Controladores y servicios reciben sus dependencias por constructor; el contenedor se configura en <code>Program.cs</code> .
Unidad de trabajo y repositorio	<code>ContextoTripSV</code> (<code>DbContext</code>) y sus <code>DbSet</code> actúan como unidad de trabajo y repositorios.
ViewModel	Las vistas reciben modelos específicos (por ejemplo <code>SitioDetalleViewModel</code>) en lugar de entidades cuando combinan datos de varias fuentes.
Resultado (Result)	Los servicios devuelven <code>Resultado.Ok(...)</code> o <code>Resultado.Error(...)</code> en vez de lanzar excepciones por errores esperables.
Configuración por entidad	Cada entidad tiene su clase <code>IEntityTypeConfiguration</code> ; el contexto las aplica todas con <code>ApplyConfigurationsFromAssembly</code> .
Vistas parciales	<code>_TarjetaSitio</code> , <code>_TarjetaCategoria</code> , <code>_Estrellas</code> y <code>_Mensajes</code> reutilizan marcado en varias páginas.
S – Responsabilidad única	Cada servicio atiende un solo módulo; la validación de imágenes y el saneamiento de HTML están en clases separadas.
O – Abierto/cerrado	Agregar una entidad nueva no requiere modificar la configuración de las existentes.
L – Sustitución de Liskov	Las pruebas sustituyen <code>ContextoTripSV</code> por un contexto derivado sobre SQLite sin cambiar los servicios.

I – Segregación de interfaces	Interfaces pequeñas por módulo en lugar de un servicio general.
D – Inversión de dependencias	Los controladores dependen de interfaces (<code>ISitiosServicio</code>), no de implementaciones.

3. Tecnologías utilizadas

Componente	Tecnología	Versión
Plataforma	.NET (SDK)	10.0.400
Framework web	ASP.NET Core MVC con Razor	10.0
Acceso a datos	Entity Framework Core (proveedor SQL Server)	10.0.12
Base de datos	Microsoft SQL Server (compatible con Express y LocalDB)	2019 o superior
Autenticación	ASP.NET Core Identity con Entity Framework	10.0.12
Saneamiento de HTML	HtmlSanitizer (Ganss.Xss)	9.2.1039
Interfaz	Bootstrap · Bootstrap Icons · Google Fonts (Poppins, Inter)	5.3.3 · 1.11.3
Scripts de cliente	jQuery · jQuery Validation Unobtrusive	3.7.1 · 1.21.0
Editor de contenido	CKEditor	4.14.0
Pruebas	xUnit · EF Core SQLite en memoria	2.9.3 · 10.0.12
Documentación	Microsoft Playwright (capturas y PDF)	1.x

4. Estructura del código

TripSV.slnx	
TripSV/	Proyecto web
— Program.cs	Configuración de servicios y canalización HTTP
— appsettings.json	Cadena de conexión y usuarios iniciales
— Modelos/	Entidades: Categoria, Sitio, Comentario, Puntuacion, Favorito, Itinerario, Visita, Usuario, Roles
— Datos/	
— ContextoTripSV.cs	DbContext (incluye tablas de Identity)
— Configuraciones/	Fluent API: tablas, columnas, índices y relaciones
— Migraciones/	MigracionInicial, AgregaFavoritos, AgregaItinerarios
— Migracion/	Script de carga del catálogo inicial
— SembradorDatos.cs	Aplica migraciones y crea roles y usuarios iniciales
— ExtensionesContexto.cs	Guardado con manejo de conflictos
— Servicios/	Reglas de negocio (interfaz + implementación)
— ViewModels/	Modelos de vista y formularios
— Controllers/	Inicio, Categorías, Sitios, Comentarios, Puntuaciones, Favoritos, Itinerarios, Cuenta
— Areas/Administracion/	Categorías, Sitios y Moderacion (rol Administrador)
— Views/	Vistas Razor y parciales compartidas
— wwwroot/	CSS, JavaScript, imágenes y librerías
TripSV.Pruebas/	Pruebas automatizadas (xUnit)
Herramientas/	Utilidades: generación de documentación y del script de datos
Documentacion/	Manuales en PDF y sus fuentes
Ejecutable/	Aplicación publicada lista para ejecutar

4.1 Controladores y rutas principales

Controlador	Acceso	Responsabilidad
InicioController	Público	Página de inicio, página de error y páginas de estado (404, 403).
CategoriasController	Público	Listado de categorías e imagen de cada categoría.
SitiosController	Público	Catálogo con búsqueda y filtros, destinos por categoría, detalle e imagen de cada destino.
CuentaController	Público	Registro, inicio y cierre de sesión, recuperación de contraseña y acceso denegado.
ComentariosController	Usuario	Publicar reseñas y respuestas, eliminar las propias.
PuntuacionesController	Usuario	Calificar un destino.
FavoritosController	Usuario	Guardar y quitar destinos favoritos, listar los propios.

<code>ItinerariosController</code>	Usuario	Crear, editar y eliminar itinerarios; agregar, ordenar, mover y quitar visitas.
<code>Administracion/Categorias</code>	Administrador	CRUD de categorías con imagen.
<code>Administracion/Sitios</code>	Administrador	CRUD de destinos con imagen y contenido enriquecido; eliminación múltiple.
<code>Administracion/Moderacion</code>	Administrador	Buscar, ocultar, mostrar y eliminar comentarios y respuestas.

4.2 Servicios

Servicio	Reglas principales
<code>CategoriasServicio</code>	Nombre único; no se elimina una categoría con destinos asociados; listado con conteo de destinos sin cargar imágenes.
<code>SitiosServicio</code>	Nombre único; imagen obligatoria al crear; contenido enriquecido saneado; búsqueda tolerante a tildes y mayúsculas con varias palabras.
<code>ComentariosServicio</code>	Hilos de un nivel (una respuesta a una respuesta se agrupa bajo el comentario original); solo el autor o un administrador elimina; ocultar un comentario oculta sus respuestas.
<code>PuntuacionesServicio</code>	Una calificación por usuario y destino (se actualiza si ya existe); promedio redondeado a un decimal.
<code>FavoritosServicio</code>	Alterna agregar o quitar; un favorito por usuario y destino.
<code>ItinerariosServicio</code>	Hasta 15 días por viaje y 10 destinos por día; sin duplicados en el mismo día; orden consecutivo; cada usuario solo accede a sus itinerarios.
<code>SanitizadorHtml</code>	Elimina scripts, eventos y enlaces peligrosos; solo permite <code>iframe</code> de Google Maps y YouTube.
<code>ValidadorImagen</code>	Formatos JPG, JPEG y PNG; máximo 5 MB.

5. Modelo de datos

Tabla	Entidad	Contenido	Restricciones destacadas
<code>categorias</code>	Categoría	Nombre, descripción e imagen	Nombre único
<code>sitios</code>	Sitio	Destino con imagen, ubicación, contenido enriquecido, calificación promedio y total de calificaciones	Nombre único; FK a categoría (sin borrado en cascada)
<code>comentarios</code>	Comentario	Reseñas y respuestas, autor, fecha y bandera de moderación <code>oculto</code>	FK a sitio (cascada), autorreferencia <code>respuesta_a_id</code>
<code>puntuaciones</code>	Puntuacion	Calificación de 1 a 5	Única por (sitio, usuario); CHECK 1-5
<code>favoritos</code>	Favorito	Destino guardado por un usuario	Único por (usuario, sitio)
<code>itinerarios</code>	Itinerario	Viaje: nombre, fecha de inicio, cantidad de días y notas	CHECK de 1 a 15 días
<code>visitas</code>	Visita	Destino planificado para un día del itinerario, con orden y notas	Única por (itinerario, día, sitio); CHECK de día
<code>usuarios</code> , <code>roles</code> , <code>usuarios_rol</code> y demás	Identity	Cuentas, contraseñas cifradas y roles	Tablas estándar de ASP.NET Core Identity renombradas al español

5.1 Reglas de borrado

Al eliminar...	Efecto
Un destino	Se eliminan en cascada sus comentarios, calificaciones, favoritos y visitas planificadas.
Una categoría	Se impide si tiene destinos asociados.
Un usuario	Se eliminan sus favoritos e itinerarios; sus comentarios y calificaciones se conservan con el nombre del autor.
Un itinerario	Se eliminan sus visitas.
Un comentario	El servicio elimina primero sus respuestas.

6. Acceso a datos y persistencia

6.1 Entity Framework Core Code-First

El esquema se define en C# (entidades y clases de configuración) y se materializa con migraciones. Las tablas y columnas usan nombres en español en formato `snake_case` mediante Fluent API. Al iniciar, la aplicación ejecuta `SembradorDatos`, que aplica automáticamente las migraciones pendientes y crea los roles y los usuarios iniciales si no existen.

```
dotnet ef migrations add NombreDeLaMigracion --project TripSV --output-dir Datos/Migraciones
dotnet ef database update --project TripSV
```

6.2 Cadena de conexión segura

La cadena incluida en `appsettings.json` usa **autenticación integrada de Windows** (`Trusted_Connection=True`), por lo que no contiene usuario ni contraseña. Para otros entornos se sobrescribe sin modificar archivos del repositorio:

Entorno	Mecanismo	Ejemplo
Desarrollo	User Secrets (fuera del repositorio)	<code>dotnet user-secrets set "ConnectionStrings:ConexionTripSV" "Server= ... ;" --project TripSV</code>
Producción	Variable de entorno	<code>ConnectionStrings__ConexionTripSV=Server= ... ;User Id= ... ;Password= ... ;Encrypt=True</code>

`TrustServerCertificate=True` es adecuado solo para desarrollo local. En producción use un certificado válido en SQL Server y `Encrypt=True`.

6.3 Uso eficiente de la persistencia

- **Proyecciones sin imágenes.** Los listados (inicio, catálogo, categorías, desplegables) seleccionan solo las columnas necesarias; las imágenes se sirven aparte desde `/Sitios/Imagen/{id}` con caché HTTP de una hora y carga diferida en el navegador.
- **Conteos en la base de datos.** El número de destinos por categoría se obtiene en la misma consulta, sin cargar los destinos completos.
- **Índices.** Índices únicos que además garantizan las reglas de negocio, e índices en todas las claves foráneas.
- **Valores precalculados.** La calificación promedio y el total de calificaciones se guardan en `sitios` y se recalculan solo cuando cambia una calificación.

- **Consultas sin seguimiento.** Los listados usan proyecciones o `AsNoTracking` , que no generan seguimiento de cambios en el contexto.

6.4 Datos iniciales

`TripSV/Datos/Migracion/datos-mysql.sql` carga el catálogo de arranque: categorías, destinos con sus imágenes, reseñas y calificaciones. Se ejecuta una sola vez después de crear la base de datos y se detiene si las tablas ya contienen información:

```
sqlcmd -S localhost -d TripSV -E -i TripSV/Datos/Migracion/datos-mysql.sql
```

7. Seguridad

Amenaza o requisito	Medida implementada
Autenticación	ASP.NET Core Identity. Contraseñas cifradas con PBKDF2 y sal aleatoria; mínimo 8 caracteres con al menos un número y una minúscula; bloqueo de 10 minutos tras 5 intentos fallidos.
Autorización	Roles <code>Administrador</code> y <code>Usuario</code> . <code>[Authorize(Roles = "Administrador")]</code> en todo el área de administración y <code>[Authorize]</code> en los módulos de usuario. Verificado con pruebas automatizadas.
Sesión	Cookie <code>HttpOnly</code> con <code>SameSite=Lax</code> , expiración deslizante de 90 minutos.
Falsificación de solicitudes (CSRF)	<code>[ValidateAntiForgeryToken]</code> en todas las acciones POST (una prueba automatizada recorre todos los controladores).
Inyección SQL	Todo el acceso a datos usa consultas parametrizadas de Entity Framework; no hay SQL concatenado.
Cross-site scripting (XSS)	Razor codifica toda la salida. El único HTML enriquecido (contenido de destinos) se sanea con <code>HtmlSanitizer</code> al guardar y al mostrar.
Acceso a datos ajenos	Toda consulta de favoritos e itinerarios filtra por el usuario autenticado; un identificador ajeno responde como inexistente.
Redirección abierta	Los parámetros de retorno se validan con <code>Url.IsLocalUrl</code> .
Carga de archivos	Solo JPG, JPEG o PNG de hasta 5 MB; límite de 10 MB por solicitud en los formularios de administración.
Encabezados HTTP	<code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , <code>Referrer-Policy</code> y <code>Permissions-Policy</code> en todas las respuestas; HTTPS y HSTS fuera de desarrollo.
Secretos	Cadenas de conexión y contraseñas iniciales configurables por User Secrets o variables de entorno.

8. Manejo de errores y excepciones

- **Errores de negocio** (datos inválidos, duplicados, permisos): los servicios devuelven un `Resultado` con un mensaje claro que se muestra al usuario; no se usan excepciones para el flujo normal.
- **Conflictos de base de datos** (dos operaciones simultáneas sobre un índice único o una clave foránea): `GuardarSinConflictoAsync` captura `DbUpdateException` y devuelve un mensaje amigable en vez de un error 500.
- **Validación de formularios**: `DataAnnotations` en servidor y validación en el navegador con `jQuery Validation`.
- **Páginas de estado**: `UseStatusCodePagesWithReExecute` muestra páginas propias para 404, 403 y 400.
- **Excepciones no controladas**: en producción, `UseExceptionHandler` muestra una página de error con el código de solicitud para rastrearlo en los registros; en desarrollo se muestra la página detallada de ASP.NET Core.

9. Rendimiento y escalabilidad

9.1 Rendimiento

- Consultas proyectadas e índices descritos en la sección 6.3.
- Imágenes con caché HTTP y carga diferida (`loading="lazy"`).
- Archivos estáticos con caché de 7 días; `asp-append-version` invalida la caché al cambiar un archivo.
- Vistas Razor compiladas en la publicación.

9.2 Escalabilidad

- **Aplicación sin estado en el servidor.** La autenticación y los mensajes temporales viajan en cookies cifradas, por lo que se pueden ejecutar varias instancias detrás de un balanceador de carga.
- **Al escalar horizontalmente** se deben compartir las llaves de Data Protection entre instancias (por ejemplo con `PersistKeysToFileSystem` en un recurso compartido o un almacenamiento en la nube) para que las cookies sean válidas en todas.
- **Base de datos.** SQL Server permite escalar verticalmente o agregar réplicas de lectura; el `DbContext` por solicitud admite cambiar a `AddDbContextPool` para reducir el costo de creación de contextos con alto tráfico.
- **Imágenes.** Si el catálogo crece mucho, las imágenes pueden trasladarse a un almacenamiento de objetos con CDN sin cambiar las vistas, porque todas se sirven a través de la acción `Imagen` .

10. Configuración del entorno de desarrollo

10.1 Requisitos

Herramienta	Uso
.NET SDK 10	Compilar, ejecutar y probar.
SQL Server 2019+ (Developer, Express o LocalDB)	Base de datos.
Herramienta <code>dotnet-ef</code>	Migraciones: <code>dotnet tool install --global dotnet-ef</code>
Visual Studio 2026, Visual Studio Code o Rider	Edición y depuración.
Git	Control de versiones.

10.2 Pasos

- 1 Clonar el repositorio: `git clone https://github.com/MoisesM02/TripSV.git`
- 2 Revisar la cadena `ConexionTripSV` en `TripSV/appsettings.json` o definirla con User Secrets (sección 6.2).
- 3 Crear la base de datos: `dotnet ef database update --project TripSV`
- 4 Cargar el catálogo inicial con el script de la sección 6.4.
- 5 Ejecutar la aplicación: `dotnet run --project TripSV` y abrir `http://localhost:5255`.
- 6 Ejecutar las pruebas: `dotnet test`

10.3 Usuarios iniciales

Usuario	Contraseña	Rol
administrador	Administrador123\$	Administrador
visitante	Visitante123\$	Usuario

Son credenciales de demostración definidas en `appsettings.json` (sección `UsuariosIniciales`). Cámbielas antes de cualquier despliegue real.

11. Guía de despliegue

11.1 Ejecutable incluido

La carpeta `Ejecutable/` contiene `TripSV.zip`, la aplicación publicada en modo Release para **Windows x64**, dependiente del runtime. Requiere tener instalado el **ASP.NET Core Runtime 10** y acceso a SQL Server. Se descomprime, se ajusta `appsettings.json` o las variables de entorno y se ejecuta `TripSV.exe`.

11.2 Generar una publicación

```
dotnet publish TripSV/TripSV.csproj -c Release -r win-x64 --self-contained false -p:SatelliteResourceLanguages=es -o publicacion
```

Para Linux se usa `-r linux-x64`. Con `--self-contained true` la publicación incluye el runtime y no es necesario instalarlo en el servidor, a cambio de un paquete más grande.

11.3 Configurar y ejecutar

- 1 Definir las variables de entorno:

```
ASPNETCORE_ENVIRONMENT=Production
ConnectionStrings__ConexionTripSV=Server=servidor;Database=TripSV;User
Id= ... ;Password= ... ;Encrypt=True
UsuariosIniciales__Administrador__Password=UnaClaveSegura123
```

- 2 Ejecutar con Kestrel: `dotnet TripSV.dll --urls "http://0.0.0.0:5000"`, idealmente detrás de un proxy inverso con HTTPS.
- 3 En IIS: instalar el *ASP.NET Core Hosting Bundle*, crear un sitio que apunte a la carpeta publicada y configurar el grupo de aplicaciones con "Sin código administrado".
- 4 La primera ejecución aplica las migraciones y crea los usuarios iniciales. Después, cargar el catálogo inicial si se desea.

11.4 Lista de verificación para producción

- Cambiar las contraseñas de los usuarios iniciales.
- Usar una cadena de conexión con cifrado y un usuario de base de datos con permisos mínimos.
- Publicar solo sobre HTTPS con un certificado válido.
- Configurar el almacenamiento compartido de llaves si hay más de una instancia.
- Programar respaldos de la base de datos.

12. Pruebas automatizadas

El proyecto `TripSV.Pruebas` contiene 80 casos de prueba con xUnit que cubren los componentes críticos: calificaciones, comentarios y moderación, favoritos, itinerarios, categorías y destinos, validación de imágenes, saneamiento de HTML y reglas de seguridad de los controladores. Los servicios se prueban contra una base SQLite en memoria, que es relacional y respeta índices únicos, claves foráneas y restricciones.

```
dotnet test
```

El detalle de cada prueba y sus resultados está en el **Documento de Pruebas**.

13. Flujo de trabajo con Git y GitHub

El repositorio `MoisesM02/TripSV` mantiene una rama por integrante, identificada por su carné. Los módulos de cada integrante se integran a `main` mediante Pull Requests con *merge commit*, lo que conserva en el historial qué integrante entregó cada módulo. Las ramas se crean a partir del `main` actualizado, de modo que la integración no produce conflictos.

Rama	Integrante	Aportes
<code>rc220772</code>	Alberto Ramos Cruz	Modelo de datos, contexto y migraciones de EF Core, script de datos iniciales; buscador de destinos con filtros.
<code>hc220857</code>	Rene Eduardo Hernandez Castro	Autenticación con Identity, roles y módulo de cuentas; sistema de favoritos.
<code>ma220150</code>	Moises Alonso Marroquin Ayala	Capa de servicios, catálogo público y configuración de la aplicación; planificador de viajes.
<code>rg210380</code>	Rafael Adolfo Ruiz García	Panel de administración y moderación; rediseño de la interfaz, pruebas automatizadas y documentación.

14. Cómo extender el sistema

Pasos para agregar un módulo nuevo siguiendo las convenciones del proyecto:

- 1 Crear la entidad en `Modelos/` con sus `DataAnnotations` de validación.
- 2 Crear su configuración en `Datos/Configuraciones/` (tabla, columnas en español, índices y relaciones).
- 3 Agregar el `DbSet` en `ContextoTripSV` y generar la migración.
- 4 Crear la interfaz y el servicio en `Servicios/`, devolviendo `Resultado` y guardando con `GuardarSinConflictoAsync`.
- 5 Registrar el servicio en `Program.cs`.
- 6 Crear el controlador con la autorización adecuada y `[ValidateAntiForgeryToken]` en las acciones POST.
- 7 Crear las vistas reutilizando las parciales y clases de estilo existentes.
- 8 Agregar pruebas en `TripSV.Pruebas` para las reglas del servicio.

14.1 Herramientas incluidas

Archivo	Uso
<code>Herramientas/GenerarDocumentacion.cs</code>	Con la aplicación en ejecución, toma las capturas de pantalla y genera los PDF de <code>Documentacion/</code> : <code>dotnet run Herramientas/GenerarDocumentacion.cs -- todo</code> .
<code>Herramientas/ConvertirDumpMySQL.cs</code>	Regenera el script de datos iniciales a partir del respaldo de la base de datos anterior.